


# OOP

programare orientată pe obiecte

# CURS09: OO DESIGN PRINCIPLES

Alexandru Olteanu
{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Universitatea Politehnica București

- "The ratio of time spent reading versus writing [code] is well over 10 to 1." *- Robert C. Martin "Uncle Bob" author Clean Code, Agile Manifesto*
- "Code never lies, comments sometimes do" *- Ron Jeffries, author of Extreme Programming books*
- "Clean code can be read, and enhanced by a developer other than its original author." *- Dave Thomas, godfather of the Eclipse strategy*
- "Clean code does one thing well." *- Bjarne Stoustrup, inventor of C++*
- "Clean code reads like well-written prose." *- Grady Booch, author of Object Oriented Analysis*

SOLID: five design principles synthesized by Robert C. Martin (Uncle Bob):

- SRP: Single Responsibility Principle
- OCP: Open-Closed Principle
- LSP: Liskov Substitution Principle
- ISP: Interface Segregation Principle
- DIP: Dependency Inversion Principle


alexandru.olteanu@upb.ro

SRP: Single Responsibility Principle A class should have only one responsibility, meaning that a class should have one and only one reason to change.
Robert C. Martin "The Single-Responsibility Principle"

# Example from Uncle Bob:

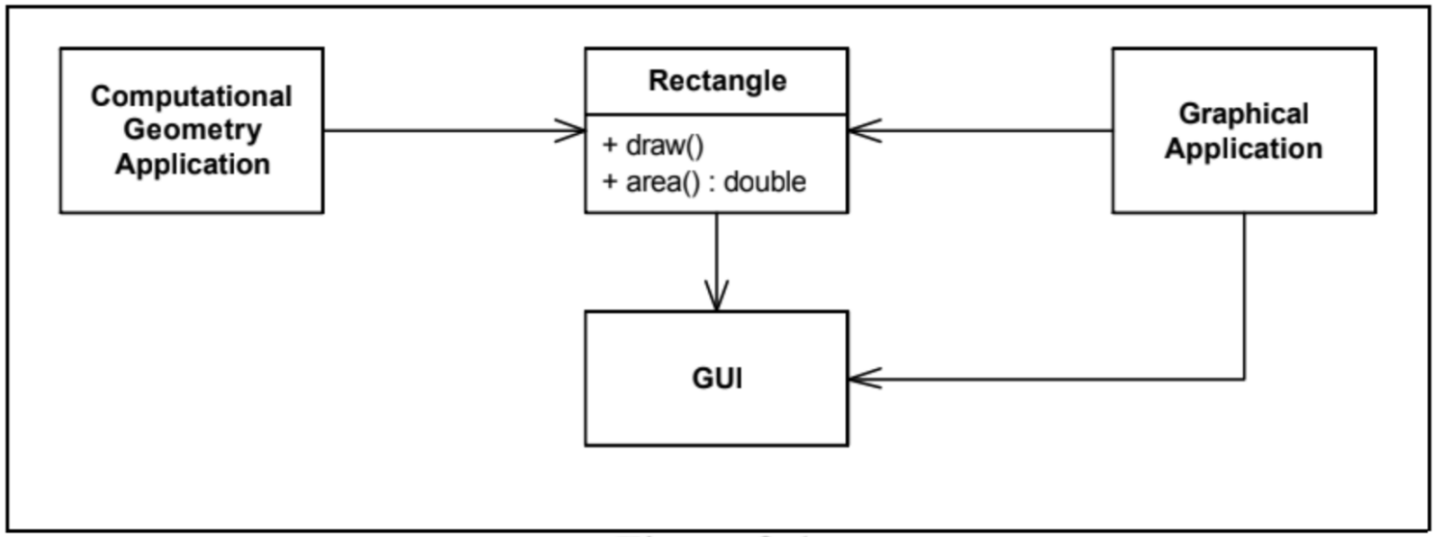



**Figure 8-1**
More than one responsibility

a Rectangle class should not contain both draw and area functions

# Opposing views

- Why SOLID Principles are not a solid solution for Software Design, towardsdatascience
- SOLID is not solid booklet


Single Responsibility Principle
Just because you *can* doesn't mean you *should*.

OCP: Open-Closed Principle You should be able to extend a classes behavior, without modifying it. Robert C. Martin "The Open-Closed Principle", C++ Report, January 1996

OCP: Open-Closed Principle Software entities should be open for extension, but closed for modication. Bertrand Meyer "Object-Oriented Software Construction", 1988

# Example from Uncle Bob:

OOD solution to Square/Circle problem.

```
class Shape
{
  public:
    virtual void Draw() const = 0;
};

class Square : public Shape
{
  public:
    virtual void Draw() const;
};

class Circle : public Shape
{
  public:
    virtual void Draw() const;
};

void DrawAllShapes(Set<Shape*>& list)
{
  for (Iterator<Shape*>i(list); i; i++)
     (*i)->Draw();
}
```

having an AbstractShape, with a generic draw method, allows us to add new shapes

# de ce extinderea s-a facut contra OCP?

```
class Person {
  private String name;
  void present() {
    System.out.println(name);
  }
}

class Doctor extends Person {
  void present() {
    System.out.println("Dr. "+name);
  }
}
```

# cum ar fi trebuit facuta extinderea conform OCP?

# extinderea conform OCP

```
class Person {
  private String name;
  void present() {
    System.out.println(getName());
  }

  String getName() {
    return name;
  }
}

class Doctor extends Person {
  String getName() {
    return "Dr. "+name;
  }
}
```

Atentie! principiul se refera la orice forma de extindere, nu doar mostenire

# Opposing views

- Why SOLID Principles are not a solid solution for Software Design, towardsdatascience
- SOLID is not solid booklet


Open-Closed Principle
Open-chest surgery isn't needed when putting on a coat.

LSP: Liskov substitution principle Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program. Robert C. Martin "The Liskov substitution principle", C++ Report

# Example from Uncle Bob:

```
Square s;
s.SetWidth(1); // Fortunately sets the height to 1 too.
s,SetHeight(2); // sets width and heigt to 2, good thing.
```

But consider the following function:

```
void f(Rectangle& r)
{
  r.SetWidth(32); // calls Rectangle::SetWidth
}
```

If we pass a reference to a `Square` object into this function, the `Square` object will be corrupted because the height won't be changed. This is a clear violation of LSP. The `f`

a square might be a rectangle, but the behavior of a Square object is not consistent with the behavior of a Rectangle object

```
class Person {
}
class Doctor extends Person {
  void performSurgery() {}
}
class Programmer extends Person {
  void writeApplication() {}
}
```

de ce putem spune ca obiectele din ierarhia Person nu sunt conforme LSP?

```
List<Person> people = ...
for (Person p : people) {
  ???
}
```

cum ar fi trebuit modelata aceasta situatie conform LSP?


alexandru.olteanu@upb.ro

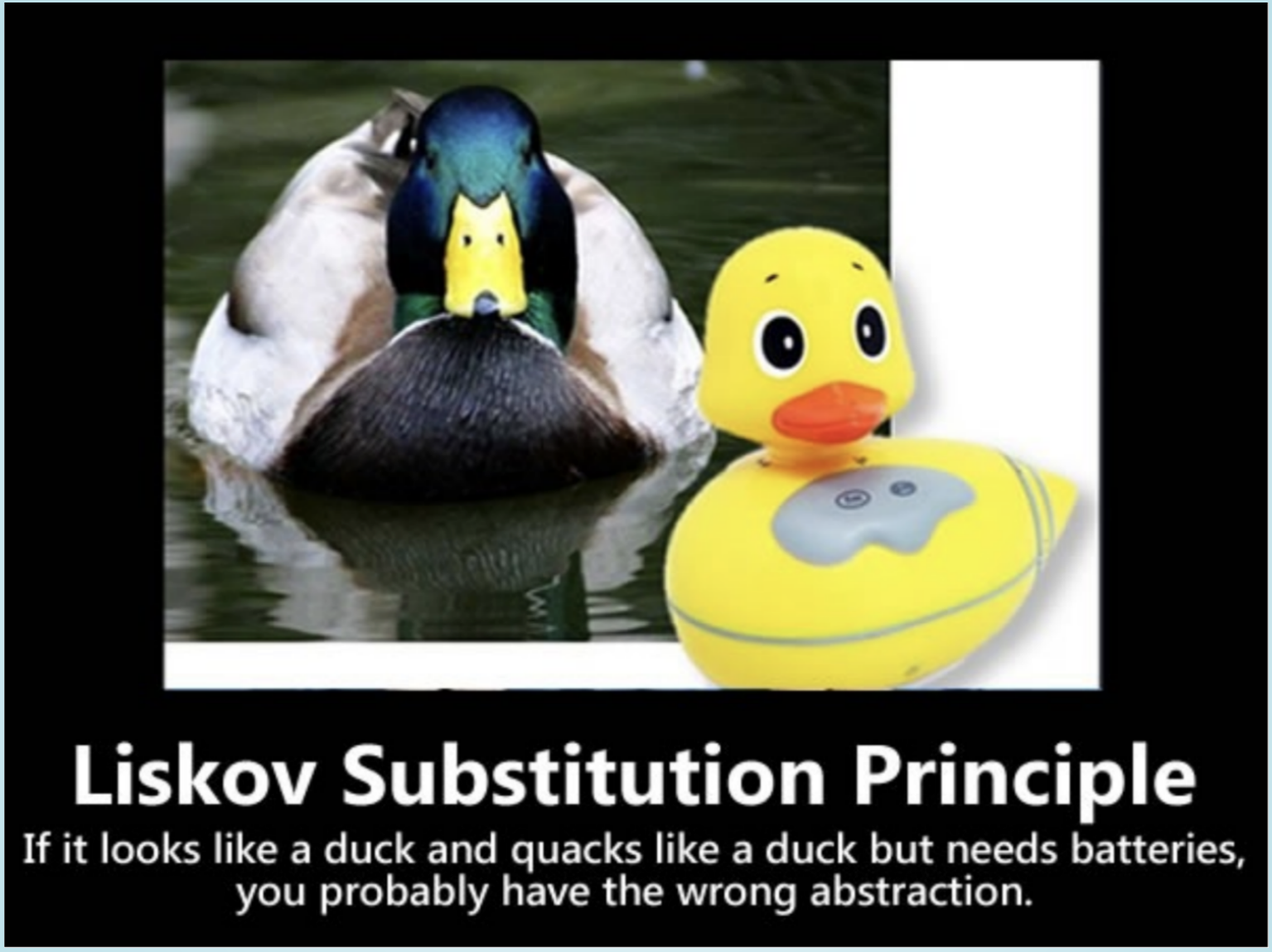
# ierarhia conform LSP

```
class Person {
  void work() {}
}
class Doctor extends Person {
  void performSurgery() {}
  void work() { performSurgery(); }
}
class Programmer extends Person {
  void writeApplication() {}
  void work() { writeApplication(); }
}

List<Person> people = ...
for (Person p : people) {
  p.work();
}
```

# Opposing views

- Why SOLID Principles are not a solid solution for Software Design, towardsdatascience
- SOLID is not solid booklet

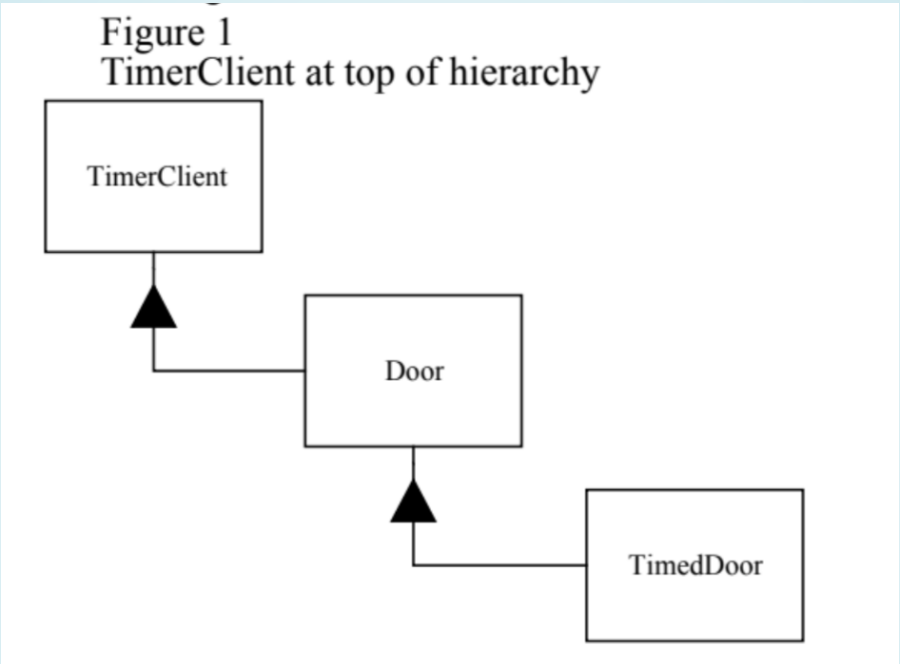





The SOLID Principles, Explained with Motivational Posters

ISP: Interface Segregation Principle Many client-specic interfaces are better than one general-purpose interface. Robert C. Martin "The Interface segregation principle"

# Example from Uncle Bob:




having a TimedDoor class, shouldn't require all Doors to have a Timer

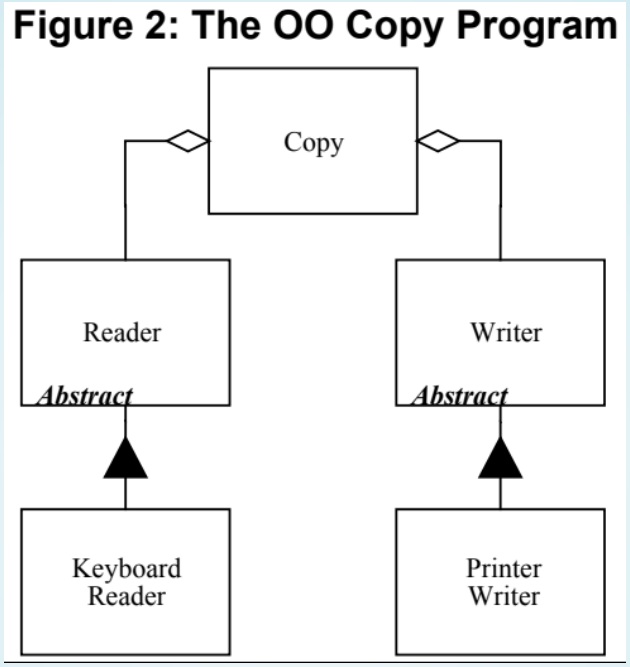
## Opposing views

- Why SOLID Principles are not a solid solution for Software Design, towardsdatascience
- SOLID is not solid booklet


Interface Segregation Principle
You want me to plug this in *where?*

DIP: Dependency Inversion Principle High level modules should not depend upon low level modules. Both should depend upon abstractions.
Robert C. Martin "The Dependency Inversion Principle", C++ Report

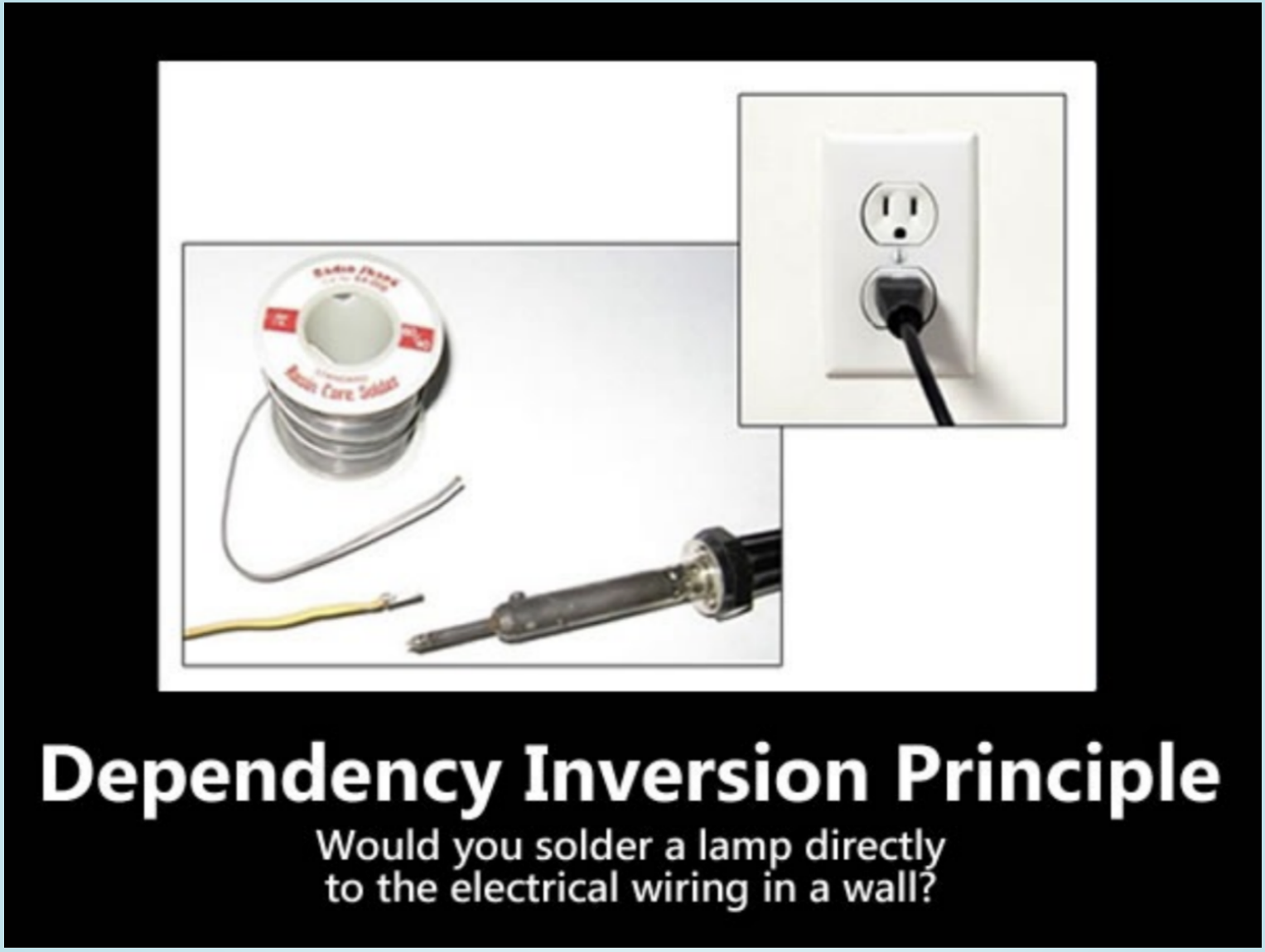
# Example from Uncle Bob:




a program that copies from inputs to outputs, should not depend on the input being a keyboard and the output being a screen

# Opposing views

- Why SOLID Principles are not a solid solution for Software Design, towardsdatascience
- SOLID is not solid booklet


Dependency Inversion Principle
Would you solder a lamp directly
to the electrical wiring in a wall?

Mai multe principii:

- Principles of OOD + package principles
- Other OO Design Principles + Law of Demeter, Betrand Meyer principles
- KISS, YAGNI, DRY: 3 Principles to Simplify Your Life as a Developer

Ar trebui sa puteti intelege tot ce scrie aici:

- LSP vs OCP discussion on stackexchange
- Why SOLID Principles are not a solid solution for Software Design, towardsdatascience
- acelasi tutorial in limbaje diferite:
  - PHP
  - JavaScript